



Research Intake 2026

Day 17

Seq2Seq Models & Attention Mechanism

A Beginner's Guide for Students

From foundational concepts to research-level understanding

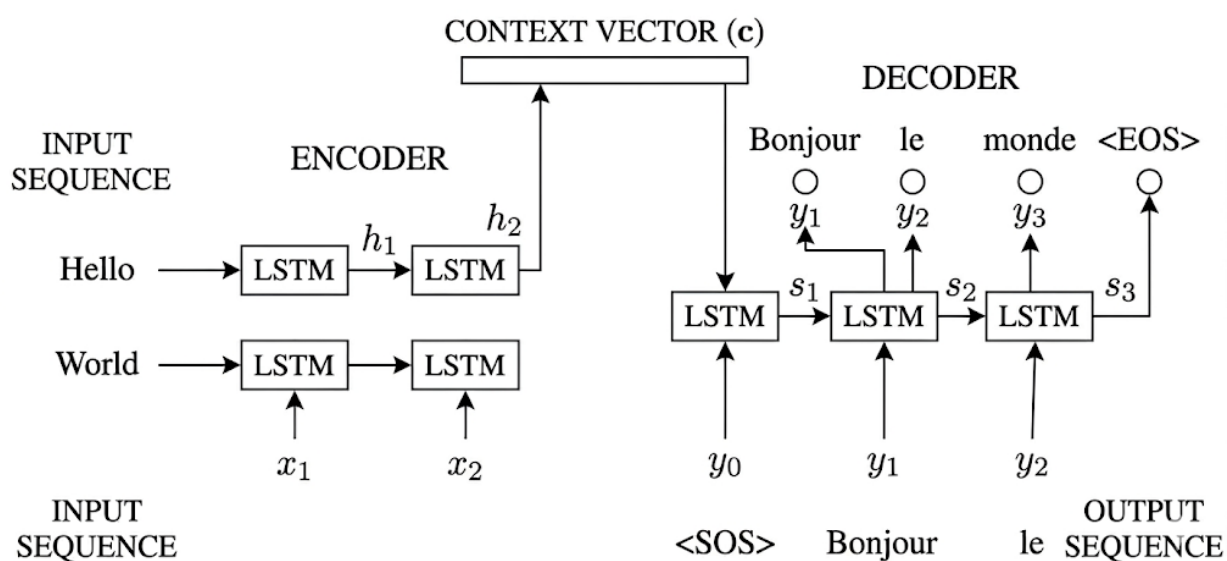
Gudsky Research Foundation

The Encoder-Decoder Framework

The encoder-decoder (Seq2Seq) architecture, introduced by Sutskever, Vinyals & Le at Google in 2014, fundamentally transformed neural machine translation and sequence modelling. It solved a longstanding challenge: how to map a variable-length input sequence to a variable-length output sequence using neural networks alone, without hand-engineered alignment tables or phrase-based statistical components.

Prior to Seq2Seq, neural MT systems were limited to fixed-length inputs or required alignment annotations. The encoder-decoder design elegantly side-stepped these constraints by compressing the entire source sequence into a single fixed-size vector — the context vector — and then conditioning a separate decoding network on that representation.

SEQUENCE-TO-SEQUENCE ENCODER-DECODER WITH LSTM ARCHITECTURE



Original Publication

Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. Advances in Neural Information Processing Systems (NeurIPS) 2014.

The paper reported a BLEU score of 34.81 on English-to-French translation — the first neural MT result competitive with phrase-based SMT.

1.1 Encoder: Compressing Input into Context

The encoder reads an input sequence $X = (x_1, x_2, \dots, x_n)$ one token at a time and produces a sequence of hidden states. In the canonical LSTM formulation, at each step t , the encoder updates:

Encoder

$$h_t = f(x_t, h_{t-1})$$

h_t is the hidden state at step t ; f is the LSTM/GRU transition function

The final hidden state h_n — sometimes called the thought vector — encapsulates the entire source sentence. This vector must carry all information needed for the decoder to regenerate meaning in the target language, a demand that becomes increasingly difficult as sequence length grows.

Practical encoders frequently employ several enhancements over the vanilla formulation:

- Bidirectional RNNs: Running a forward encoder (left-to-right) and a backward encoder (right-to-left) and concatenating their hidden states. This ensures that each position's representation is informed by its full sentential context, not just the preceding tokens.
- Multi-layer stacking: Stacking 2–4 LSTM layers with residual connections, allowing the model to learn progressively abstract representations at each layer.
- Dropout regularisation: Applied between LSTM layers (typically $p = 0.2$ – 0.5) to prevent co-adaptation and improve generalisation.
- Packed sequences: In batched training, variable-length sequences are padded to the batch maximum; packed representation avoids computing over padding tokens.

Research Note: Reversing the Input

Sutskever et al. (2014) found that reversing the input sequence before encoding significantly improved BLEU scores.

Hypothesis: Reversing places the beginning of the source sentence closer (in the unrolled computational graph) to the beginning of the target sequence, reducing the effective path length of the gradient and thus easing learning.

1.2 Decoder: Autoregressive Generation

The decoder generates the output sequence $Y = (y_1, y_2, \dots, y_m)$ autoregressively: each target token is produced conditioned on the context vector and all previously generated tokens. The decoder state s_t evolves as:

Decoder State	$s_t = g(y_{t-1}, s_{t-1}, c)$	c = context vector from encoder; g is LSTM/GRU; y_{t-1} is previous token
----------------------	--------------------------------	---

Output Prob.	$P(y_t y_{<t}, X) = \text{softmax}(W s_t + b)$	$W \in \mathbb{R}^{ V \times d}$ projects decoder state to vocabulary distribution
---------------------	--	---

At each decoding step the model effectively asks: given everything I have encoded about the source and everything I have already generated, what is the most likely next token? This probability is parameterised by the decoder hidden state, which is initialised from the final encoder state.

Three decoding strategies are commonly used at inference time:

1. Greedy decoding: Select argmax of the softmax distribution at each step. Fast, but suboptimal because early mistakes cannot be corrected.
2. Beam search: Maintain the top- k partial hypotheses at each step, expanding each and retaining the k most probable. Standard $k \in \{4, 8, 12\}$ in NMT.
3. Sampling: Draw from the distribution with optional temperature scaling. Produces diverse outputs; used in dialogue and creative generation.

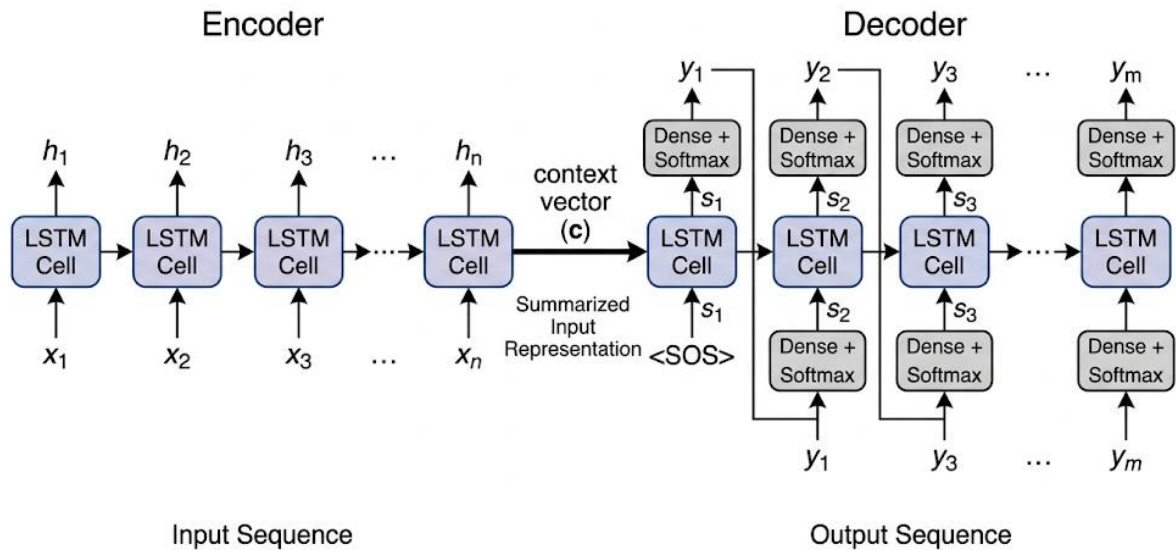
The Exposure Bias Problem

During training the decoder is fed ground-truth previous tokens (teacher forcing). At inference, it sees its own predictions.

This train-test discrepancy — called exposure bias — causes error accumulation.

Mitigations: scheduled sampling (Bengio et al., 2015) gradually replaces teacher-forced tokens with model samples during training.

LSTM-based Seq2Seq Encoder-Decoder Architecture



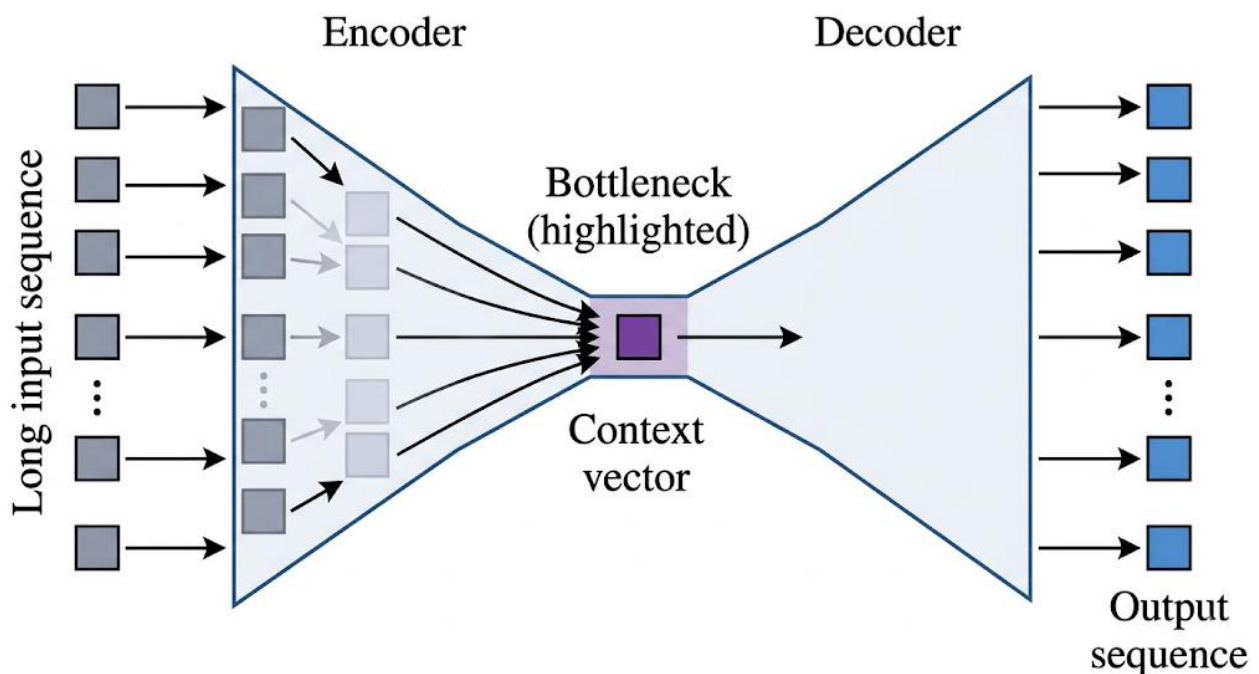
The Context Vector Bottleneck

The context vector bottleneck is the central limitation of vanilla Seq2Seq models. The entire semantic content of an arbitrarily long source sentence must be distilled into a single fixed-dimensional vector — typically 256 to 1024 dimensions — before the decoder begins generating.

The empirical consequences of this design choice are well-documented:

- Performance degrades sharply for sentences longer than ~30 tokens in the baseline formulation without attention.
- Rare and complex syntactic phenomena are poorly represented because they appear infrequently in training and must share representational capacity with more common patterns.
- Translating long-range dependencies (e.g., agreement between a verb and its head noun 15 tokens away) is unreliable.

Bahdanau, Cho, and Bengio (2015) quantified this degradation empirically, showing BLEU scores for an encoder-decoder without attention falling monotonically as sentence length increased beyond 20 tokens, while human translation quality remained constant. This observation directly motivated the attention mechanism.



Capacity Analysis

If we have a vocabulary of size V and embed tokens into $\mathbb{R}^d$, the encoder must route all contextual information through a d^H -dimensional bottleneck.

For English sentences of average length 25 tokens, the model must compress $\sim 25 \times d$ dimensions of input into d^H output dimensions.

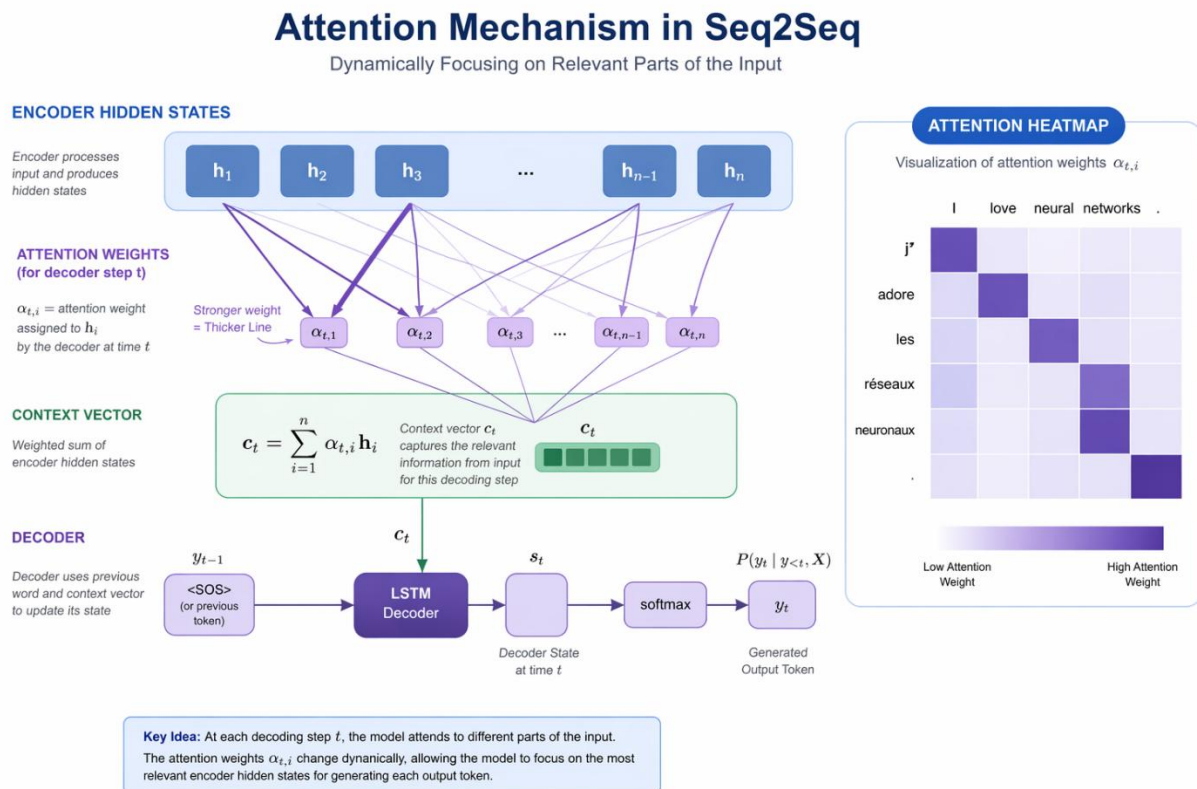
For WMT 2014 En-Fr (36M sentence pairs, avg. 30 tokens), this means a 512-dim vector must encode $30 \times 512 = 15,360$ dimensions of input context — an $\sim 30\times$ compression ratio.

The bottleneck is not simply a problem of dimensionality. It is also a problem of single-point representation: the decoder cannot selectively retrieve different parts of the source depending on which part of the target it is currently generating. The attention mechanism resolves this by making the context vector dynamic — a different weighted combination of encoder states for each decoding step.

Property	Without Attention	With Attention
Context vector	Single fixed vector h_n	Dynamic vector c_t per step
Scalability	Degrades beyond 30 tokens	Robust up to 100+ tokens
Source alignment	Implicit, uninterpretable	Explicit weight matrix α_{ti}
Memory usage	$O(1)$ context	$O(n)$ encoder states stored
First paper	Sutskever et al. 2014	Bahdanau et al. 2015

Attention Mechanism

The Attention Mechanism, introduced by Bahdanau, Cho & Bengio in 2015, fundamentally changed the Seq2Seq paradigm. Instead of relying solely on the final encoder hidden state, the decoder now has access to a dynamic summary of the entire encoder hidden-state sequence — a different summary for every decoding step.



The key insight is alignment: when generating the French word 'zone' the decoder should attend most strongly to the English word 'area'. Attention learns these soft alignment distributions without supervision — purely from the gradient signal of the translation objective.

💡 Intuition: The Human Reading Analogy

When a human translator works sentence-by-sentence, they do not memorise the entire source and then generate the target from memory.

Instead, they re-read relevant parts of the source as they write each target word.

Attention gives the model the same capability: at each decoding step it can 'look back' at any part of the source.

3.1 Bahdanau (Additive) Attention

Bahdanau attention (also called additive attention) computes a score between the current decoder state s_t and each encoder hidden state h_i using a learned feed-forward network. The process proceeds in three steps:

Step 1 — Score computation: For each encoder position i , compute an unnormalised alignment score:

Score	$\mathbf{e}_{ti} = \mathbf{v}^T \tanh(\mathbf{W}_a \mathbf{s}_{t-1} + \mathbf{U}_a \mathbf{h}_i)$	$\mathbf{W}_a \in \mathbb{R}^{d \times d}, \mathbf{U}_a \in \mathbb{R}^{d \times d}, \mathbf{v}^T \in \mathbb{R}^{1 \times d}$ are learned
-------	---	--

Step 2 — Normalisation: Apply softmax across all source positions to obtain attention weights (a probability distribution):

Weights	$\alpha_{ti} = \exp(\mathbf{e}_{ti}) / \sum_j \exp(\mathbf{e}_{tj})$	$\alpha_{ti} \in [0, 1], \sum_i \alpha_{ti} = 1$ (soft alignment)
---------	--	---

Step 3 — Context vector: Compute a weighted sum of all encoder hidden states:

Context	$\mathbf{c}_t = \sum_i \alpha_{ti} \mathbf{h}_i$	$\mathbf{c}_t \in \mathbb{R}^{d_H}$ — a step-specific source summary
---------	--	--

The context vector $\mathbf{c}_t$ is then concatenated with the previous decoder hidden state and fed into the decoder LSTM, replacing the fixed context vector of the vanilla model.

- Complexity: $O(n \times m)$ attention score computations for a source of length n and target of length m .
- Parameters added: Three new weight matrices $\mathbf{W}_a, \mathbf{U}_a, \mathbf{v}$ — typically adding $\sim 3d^2$ parameters.
- Interpretability: The matrix $\{\alpha_{ti}\}$ provides a visual alignment grid that can be inspected to understand model behaviour.

Key Finding from Bahdanau et al. (2015)

On WMT English-to-French (En-Fr), the attention model achieved 28.45 BLEU with a single model — comparable to the phrase-based Moses baseline (33.3) — a remarkable result for a purely neural approach at the time. More importantly, the model showed no performance degradation on long sentences, confirming that attention addresses the bottleneck problem.

3.2 Luong (Multiplicative) Attention

Luong et al. (2015) proposed simplified attention variants that replaced Bahdanau's additive scoring with multiplicative (dot-product) operations. Three score functions were proposed and compared:

Dot	$\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \mathbf{s}_t^T \mathbf{h}_i$	No parameters — requires equal dimensions
-----	--	---

General	$\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \mathbf{s}_t^T \mathbf{W}_a \mathbf{h}_i$	$\mathbf{W}_a \in \mathbb{R}^{d \times d}$ — bilinear with learnable matrix
---------	---	---

Concat	$\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \mathbf{v}^T \tanh(\mathbf{W}_a [\mathbf{s}_t; \mathbf{h}_i])$	Equivalent to Bahdanau but applied to current $\mathbf{s}_t$
--------	--	--

Luong attention also differed architecturally: rather than using $\mathbf{s}_{t-1}$ (previous step) as in Bahdanau, it used the current decoder state $\mathbf{s}_t$ to compute attention, and fed the attended context vector into a separate attentional hidden state:

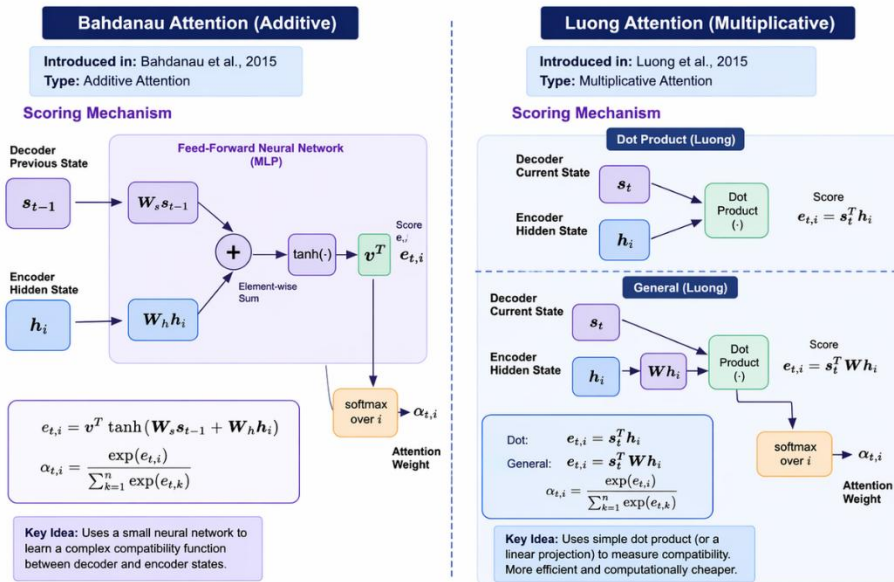
Attentional	$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_c [\mathbf{c}_t; \mathbf{s}_t])$	$\tilde{\mathbf{h}}_t$ is the final representation used for prediction
-------------	---	--

Property	Bahdanau (2015)	Luong (2015)
Score type	Additive (MLP)	Dot / General / Concat
Query	Previous state $\mathbf{s}_{t-1}$	Current state $\mathbf{s}_t$

Parameters	W_a, U_a, v	W_a (General only)
Computation	Heavier (tanh)	Lighter (dot product)
Context usage	Fed into decoder LSTM	Attentional hidden state $\tilde{h}_t$
BLEU (En-De)	~20.6	~25.9 (best)

Bahdanau vs. Luong Attention

Two Popular Attention Mechanisms in Seq2Seq Models



Aspect	Bahdanau (Additive)	Luong (Multiplicative)
Scoring Function	$e_{t,i} = v^T \tanh(W_s s_{t-1} + W_h h_i)$	$e_{t,i} = s_t^T h_i$ or $s_t^T W h_i$
Uses Previous or Current Decoder State?	Uses Previous State s_{t-1}	Uses Current State s_t
Parameters	W_s, W_h, v (more parameters)	None (dot) or W (general)
Computational Complexity (for each score $e_{t,i}$)	$O(d_a)$ with non-linear tanh	$O(d)$ for dot product (faster)
When to Use?	When you want a more expressive and flexible attention model.	When you need a simpler, faster, and scalable attention mechanism.

3.3 Attention Weights as Alignment

One of the most valuable properties of attention is its interpretability as a soft alignment matrix. Each row α_t of the weight matrix corresponds to a decoding step; each column α_i corresponds to an encoder position. The resulting grid visually reveals which source tokens the model focused on when generating each target token.

Empirical alignment patterns observed in trained NMT models include:

- **Diagonal alignment:** For closely related language pairs (e.g., French-Spanish), attention weights tend to cluster near the main diagonal, reflecting word-order similarity.
- **Monotonic alignment:** Some language pairs show near-monotonic attention — each target word attends to roughly the corresponding source position.
- **Non-monotonic alignment:** Language pairs with very different word orders (e.g., English-Japanese, English-German with verb-final clauses) show substantially non-diagonal patterns.

- Diffuse attention: Function words, punctuation, and sentence-initial tokens sometimes accumulate diffuse attention from many target positions — suggesting the model uses them as 'anchor points'.

Real-World Example: GNMT Alignment

Wu et al. (2016) visualised attention in Google NMT for the sentence pair:

EN: 'The agreement on the European Economic Area was signed in August 1992.'

FR: 'L'accord sur la zone économique européenne a été signé en août 1992.'

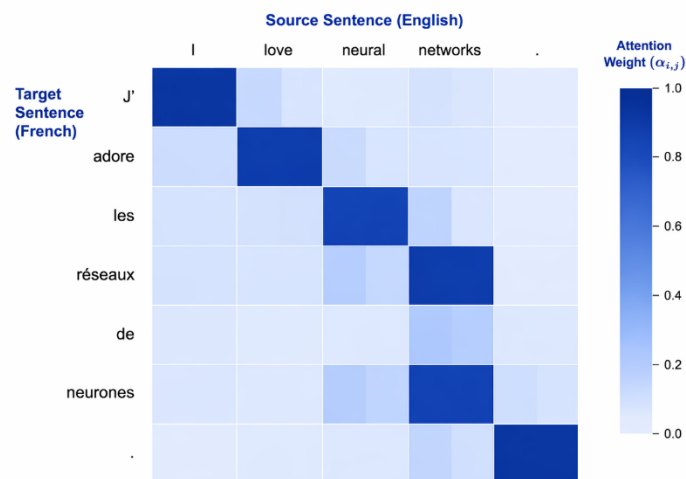
The attention matrix showed clean near-diagonal alignment with a visible swap for 'Economic Area' → 'zone économique' (adjective-noun inversion in French), confirming the model learned linguistically valid alignments without supervision.

Applications: Machine Translation & Summarisation

The combination of the Seq2Seq framework and attention mechanism unlocked a generation of high-performance sequence transduction models. This section surveys the two flagship applications that drove adoption — neural machine translation (NMT) and abstractive summarisation — together with the key design decisions that enabled production deployment.

Attention Alignment in Machine Translation

Visualization of Attention Weights Between Source and Target Words



Key Insight: Darker cells indicate higher attention weights, showing which source words the model focuses on when generating each target word.

4.1 Neural Machine Translation

Neural Machine Translation (NMT) is the task of translating a sentence from a source language to a target language using an end-to-end neural network. Prior to the Seq2Seq era, the dominant paradigm was phrase-based statistical MT (PBSMT), which required hand-engineered features, separate phrase tables, and language models. PBSMT systems were brittle, expensive to build, and difficult to extend to new language pairs.

Seq2Seq with attention achieved three things simultaneously that PBSMT required separate components for:

- Lexical translation: Mapping source words/phrases to target equivalents.
- Reordering: Placing target words in the correct order even when source order differs.
- Fluency: Producing grammatically well-formed target sentences.

Google Neural Machine Translation (GNMT, 2016)

Wu et al. (2016) deployed an 8-layer encoder, 8-layer decoder LSTM with attention, trained on ~36M sentence pairs.

For English-French, human-rated quality improved by 60% of the gap between old MT and professional human translation.

For English-Chinese, the gap closed by 58%. The system replaced Google Translate overnight for all major language pairs.

Key innovations: Residual connections between LSTM layers; wordpiece tokenisation; length-normalised beam search.

The BLEU metric (Bilingual Evaluation Understudy) is the standard automatic evaluation metric for NMT. It measures n-gram overlap between system output and reference translations. While imperfect, BLEU improvements of ≥ 1 point are generally considered significant.

System	En-Fr BLEU	En-De BLEU
PBSMT (Moses, 2014)	33.3	20.8
Seq2Seq no attn (2014)	34.8	—
Bahdanau attn (2015)	38.9	—
Luong attn (2015)	40.4	25.9
GNMT (2016)	41.2	26.3
Transformer (2017)	41.8	28.4

4.2 Abstractive Summarisation

Text summarisation aims to produce a shorter version of a document that preserves its key information. Extractive summarisation selects and concatenates existing sentences. Abstractive summarisation generates novel sentences that may not appear verbatim in the source — closer to how humans write summaries.

Nallapati et al. (2016) applied attention-based Seq2Seq to CNN/DailyMail news articles. Their model learned to generate abstractive summaries that combined information from multiple source sentences, used paraphrase, and introduced novel phrasing not present in the source.

Use Case: Abstractive Summarisation in Production

Nallapati et al. (2016) trained on CNN/DailyMail: ~300K news articles with associated highlight bullet points.

Key challenge: rare proper nouns (e.g., 'Qarabag FK') cannot be learned from training data.

Solution: Pointer network extension (see Section 5) allowing the model to copy source tokens verbatim.

Downstream impact: Apple News summary cards; Google Search featured snippets; LinkedIn article previews.

Summarisation introduces unique challenges beyond translation:

- Content selection: Which information from a 500-word article should appear in a 50-word summary?
- Compression: Multiple source sentences may need to be fused into one target sentence.
- Abstraction: The model must generate phrases not present in the source.
- Faithfulness: Hallucinating facts not present in the source is a critical failure mode.

The tension between fluency (well-formed language) and faithfulness (accurate content) remains an active research challenge, driving work on constrained decoding, fact-checking post-processing, and faithfulness metrics such as FactScore.

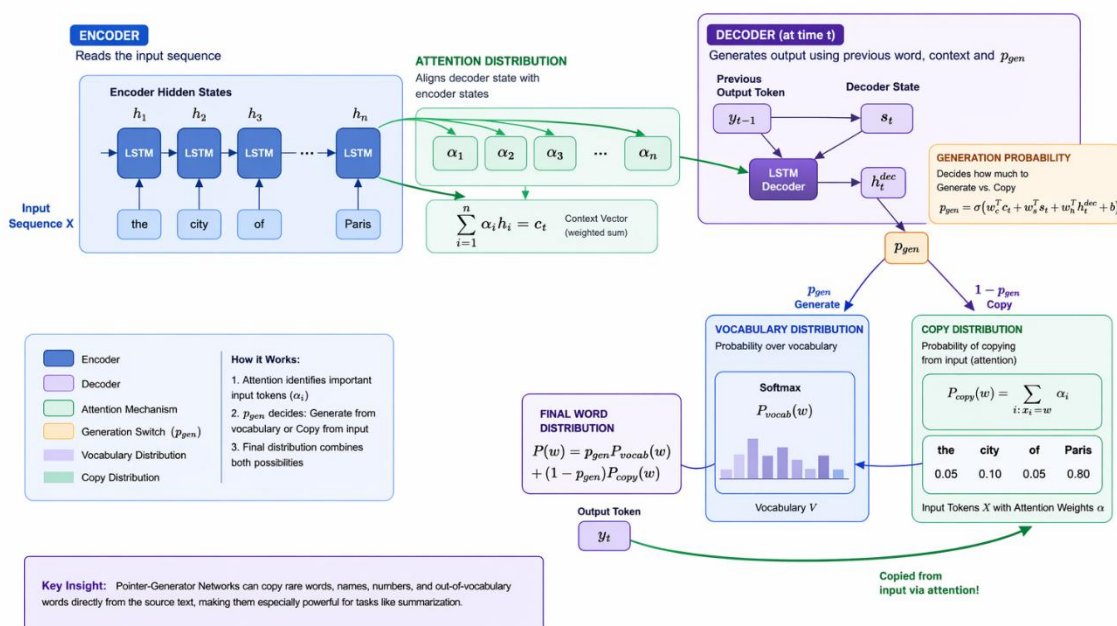
Pointer Networks and Copy Mechanism

Pointer Networks, introduced by Vinyals, Fortunato & Jaitly (Google, 2015), address a fundamental limitation of standard Seq2Seq models: the output vocabulary is fixed at training time. Any word not in the vocabulary — out-of-vocabulary (OOV) tokens — must be represented by a generic <UNK> symbol, causing information loss.

The Pointer Network resolves this by allowing the decoder to either generate a token from the fixed vocabulary or directly copy a token from the input sequence by pointing to its position. The model thus operates over both a fixed vocabulary and the dynamic input set.

Pointer-Generator Network

Seq2Seq with Copy Mechanism



5.1 The Copy Mechanism

The copy (or pointer-generator) mechanism, formalised by See, Liu & Manning (2017) in their Pointer-Generator Network, introduces a soft binary switch p_{gen} that balances generation from vocabulary versus copying from source:

Switch

$$p_{gen} = \sigma(w^H c_t + w^s s_t + w^x x_t + b_{ptr})$$

$p_{gen} \in [0, 1]$; σ is sigmoid; w^H, w^s, w^x are learned vectors

Final Prob.

$$P(w) = p_{gen} \cdot P_{vocab}(w) + (1 - p_{gen}) \cdot \sum_{i: x_i = w} \alpha_i$$

Mixture of generation probability and copy probability

When the model decides to copy ($p_{gen} \approx 0$), it effectively routes the attention distribution over source positions as a probability distribution over target tokens. This allows proper nouns, numbers, technical terms, and rare words to be faithfully reproduced even if they were never seen during training.

💡 Worked Example: Named Entity Copy

Source: 'Prime Minister Narendra Modi met with President Emmanuel Macron in Paris.'

Without copy: '... met with President <UNK> in Paris.' — Macron is OOV.

With copy: $p_{\text{gen}} \approx 0.05$ when generating 'Macron'; the model copies it from position 7 of the source.

Result: All named entities, dates, and technical terms are reproduced exactly.

5.2 Applications of Pointer Networks

Pointer networks have proven useful far beyond summarisation:

- Code generation: Copying variable names and API identifiers from specifications into generated code.
- Question answering: Extractive QA (e.g., SQuAD) can be framed as pointing to the answer span start and end.
- Combinatorial optimisation: The original Vinyals et al. (2015) paper applied pointer networks to the Travelling Salesman Problem, outputting a permutation of input cities.
- Semantic parsing: Copying tokens from natural language queries into structured query languages (SQL, SPARQL).
- Dialogue systems: Copying slot values from user utterances into system responses.

Model	Copy Mechanism	OOV Handling
Vanilla Seq2Seq	None	<UNK> token
Pointer Network (2015)	Hard pointer	Copy or generate
Pointer-Generator (2017)	Soft switch pgen	Mixture distribution
Copy Net (2016)	Attention-based copy	Selective replication

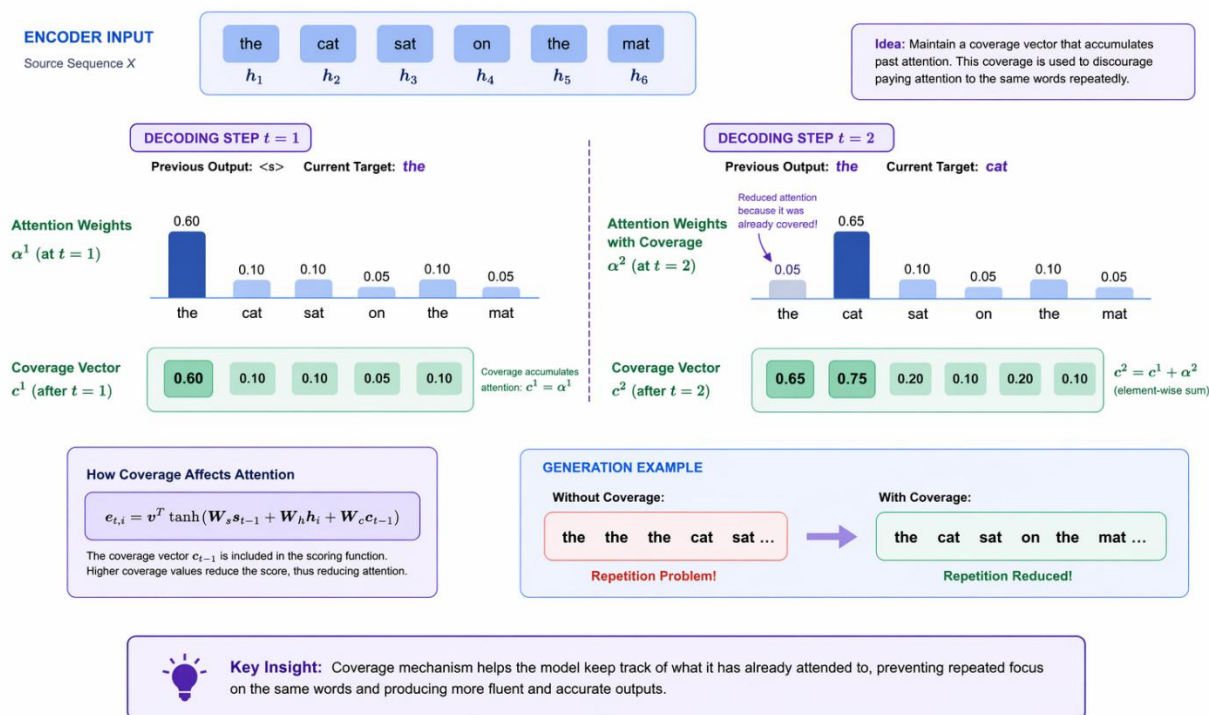
Coverage Mechanism: Handling Repetition

A persistent failure mode of early attention-based Seq2Seq models — especially in summarisation — is repetition: the model generates the same phrase multiple times in its output. This occurs because the attention mechanism at each step selects source positions independently, with no memory of which positions it has already focused on.

The Coverage Mechanism, proposed by Tu et al. (2016) for NMT and integrated into summarisation by See et al. (2017), addresses this by maintaining a coverage vector that accumulates past attention weights. The decoder is explicitly penalised for attending to already-covered source positions.

Coverage Mechanism in Attention

Preventing Repetition by Remembering What Has Already Been Attended



6.1 Coverage Vector

The coverage vector cov_t is defined as the sum of all attention weight vectors from previous decoding steps:

Coverage	$\text{cov}_t = \sum_{t' < t} \alpha_{t'}$	$\text{cov}_t \in \mathbb{R}^n$ — accumulated attention over source
-----------------	--	---

The coverage vector is fed as an additional input to the attention scoring function, modifying the score computation:

Cov. Score	$e_{ti} = v^T \tanh(W_a s_t + U_a h_i + W_{\text{cov}_{t,i}})$	$W_{\text{cov}_{t,i}} \in \mathbb{R}^d$ penalises previously attended positions
-------------------	--	---

6.2 Coverage Loss

In addition to modifying the attention score, a coverage loss encourages the model to cover each source position at most once:

Cov. Loss	$L_{cov} = \sum_i \min(a_{ti}, cov_{ti})$	Penalises attending to positions already covered
------------------	---	--

The total loss is a weighted sum of the standard cross-entropy loss and the coverage loss:

Total Loss	$L = L_{ce} + \lambda L_{cov}$	$\lambda \in \{0.1, 1.0\}$ — coverage loss weight hyperparameter
-------------------	--------------------------------	--

Impact of Coverage on Summarisation Quality

See et al. (2017) Pointer-Generator + Coverage on CNN/DailyMail:

ROUGE-1: 39.53 (vs. 36.44 without coverage) — +3.09 points

ROUGE-2: 17.28 (vs. 15.66 without coverage) — +1.62 points

ROUGE-L: 36.38 (vs. 33.42 without coverage) — +2.96 points

Qualitative: Repetition nearly eliminated from model outputs.

Coverage has since been incorporated into many production summarisation systems. Beyond simple repetition prevention, the mechanism also addresses under-coverage (important source sentences ignored entirely), leading to more balanced and complete summaries.

Limitations of Coverage

Coverage assumes each source position should be attended roughly once — a reasonable assumption for summarisation but not for all tasks.

In translation, multi-word source phrases sometimes need to be attended multiple times (for multi-word target expressions).

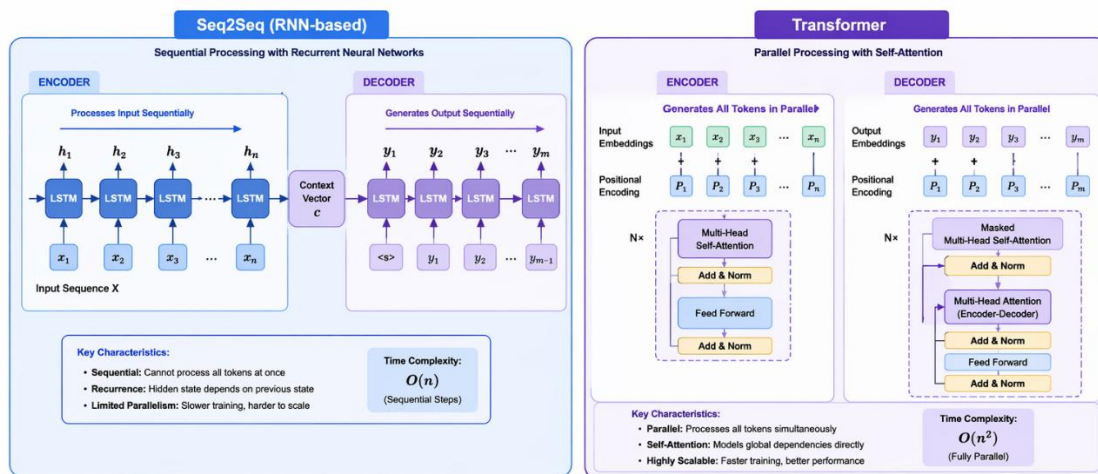
Alternatives: hierarchical attention, document-level attention, and self-attention-based coverage in Transformers.

Research Frontiers: Neural Machine Translation Milestones

This section surveys the research arc from the original Seq2Seq paper through attention-based NMT to the Transformer era and beyond. Each milestone represents either a new capability, a new scale of deployment, or a new architectural insight that shaped the field.

Seq2Seq (RNN-based) vs. Transformer

Evolution of Sequence Modeling



Aspect	Seq2Seq (RNN-based)	Transformer
Processing	Sequential	Parallel
Core Mechanism	Recurrence (LSTM/GRU)	Self-Attention (Multi-Head Attention)
Dependency Modeling	Limited by sequence length	Global dependencies, direct connections
Training Speed	Slower	Faster
Scalability	Harder to scale to long sequences	Highly scalable, state-of-the-art performance

Key Takeaway:
Transformers replaced RNN-based Seq2Seq by enabling **parallel processing** and using self-attention, leading to faster training and superior performance on long sequences.

7.1 Milestones Timeline

Year	Paper / System	Key Innovation
2014	Sutskever et al.	Seq2Seq; LSTM encoder-decoder; input reversal
2015	Bahdanau et al.	Additive attention; soft alignment; dynamic context
2015	Luong et al.	Multiplicative attention; global vs. local attention
2015	Vinyals et al.	Pointer networks; combinatorial optimisation with NNs
2016	Wu et al. (GNMT)	8-layer residual LSTM; wordpiece; production deployment
2016	Tu et al.	Coverage mechanism for NMT
2016	Nallapati et al.	Seq2Seq for abstractive summarisation
2017	See et al.	Pointer-Generator + Coverage for summarisation
2017	Vaswani et al.	Transformer: self-attention replaces recurrence entirely
2019	Child et al.	Sparse Transformer: $O(n\sqrt{n})$ for long sequences

2020	Beltagy et al.	Longformer: linear attention for documents
2020	Zaheer et al.	BigBird: random + window + global attention

7.2 The Transformer Transition

The Transformer architecture (Vaswani et al., 2017) — 'Attention Is All You Need' — replaced recurrent connections entirely with self-attention layers. This had profound consequences:

- Parallelisation: Without sequential RNN dependencies, the entire sequence can be processed in parallel, dramatically reducing training time.
- Long-range dependencies: Self-attention connects all pairs of positions in $O(1)$ steps, regardless of distance.
- Scalability: The fully-parallelisable architecture enabled training on orders of magnitude more data, leading directly to GPT, BERT, and the large language model era.

However, the Transformer retained the core attention insight from Bahdanau and Luong: dynamic, learned, weighted aggregation of representations. The specific form — query-key-value dot-product attention — is a natural extension of Luong's general attention to a multi-headed, self-attentive setting.

Self-Attention	$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k}) \mathbf{V}$	$\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{n \times d}$; d_k scaling prevents vanishing gradients
----------------	---	---

Research Spotlight: Sparse Transformer (Child et al., 2019)

Standard Transformer attention is $O(n^2)$ in sequence length, making it impractical for sequences $> 2,000$ tokens. Sparse Transformer restricts each token to attend to local windows (stride-1) and strided global positions. Complexity: $O(n\sqrt{n})$ — enabling sequences of 12,000 steps (images as pixel sequences). This directly inspired Longformer (2020) and BigBird (2020), which extended transformers to full documents.

7.3 Evaluation Metrics for NMT and Summarisation

Evaluating sequence generation systems is a non-trivial problem. Automatic metrics serve as efficient proxies for human judgement, but each has known limitations:

Metric	Translation	Summarisation
BLEU	Primary automatic metric; n-gram precision	Used but less reliable than for MT
ROUGE-N	Rarely used	Primary metric; recall-oriented n-gram overlap
ROUGE-L	Rarely used	Longest common subsequence F_1
METEOR	Handles synonyms & stemming	Used in some benchmarks
BERTScore	Embedding-based semantic similarity	Increasingly preferred
Human eval	MQM / DARR scales	Relevance, coherence, faithfulness

7.4 Open Research Problems

Despite dramatic progress, several fundamental challenges remain open:

- Hallucination and faithfulness: Models generate fluent but factually incorrect content. This is particularly dangerous in medical, legal, and news summarisation.
- Low-resource MT: Most of the world's ~7,000 languages lack sufficient parallel data for standard NMT. Zero-shot and few-shot cross-lingual transfer are active research directions.
- Document-level translation: Handling coreference, discourse coherence, and tense consistency across sentence boundaries.
- Efficiency at scale: Inference latency of large Transformer models remains prohibitive for real-time applications on edge devices.
- Controllable generation: Specifying target length, style, formality, or domain for generated text without sacrificing quality.

Frontier Research: Non-Autoregressive Generation

Standard Seq2Seq decoding is inherently sequential — each token depends on the previous one.

Non-autoregressive models (Gu et al., 2018; BANG, Qi et al., 2021) generate all target tokens in parallel.

Speedup: 2–15× faster inference, at some cost to BLEU (typically 1–3 points behind autoregressive).

Applications: Real-time translation APIs, on-device MT, low-latency voice translation.

The research frontier has also moved significantly toward multimodal and multilingual unification. Models like mBART, mT5, and NLLB-200 (Meta, 2022) train a single Seq2Seq model across 200+ languages, using cross-lingual transfer to achieve competitive performance even on extremely low-resource pairs such as Acehnese-English and Tamasheq-French.

Summary & Key Takeaways

Day 17 has covered the full arc of Seq2Seq models and attention, from the foundational encoder-decoder architecture through the attention mechanism and its variants to production deployments and cutting-edge research directions. The following synthesises the most important insights:

- The encoder-decoder paradigm (Sutskever et al., 2014) established the template for neural sequence transduction: compress input into context, then decode autoregressively.
- The context vector bottleneck limits vanilla Seq2Seq to short sequences; performance degrades sharply beyond ~30 tokens without attention.
- Bahdanau (additive) attention (2015) resolves the bottleneck with a dynamic, step-specific context vector $c_t = \sum_i \alpha_{ti} h_i$, while also providing interpretable alignment.
- Luong (multiplicative) attention (2015) simplifies scoring with dot-product operations and shows stronger empirical BLEU on standard NMT benchmarks.
- Attention weights are interpretable as soft alignment matrices, revealing linguistically meaningful source-target correspondences.
- Pointer networks and the copy mechanism (2015–2017) enable faithful copying of OOV tokens — critical for named entity handling in summarisation.
- Coverage mechanism (2016–2017) prevents repetition by accumulating past attention and penalising re-attendance.
- The Transformer (2017) generalised attention to self-attention, removing recurrence and enabling full parallelisation — the direct ancestor of all large language models.
- Sparse attention variants (Sparse Transformer, Longformer, BigBird) reduce complexity from $O(n^2)$ to $O(n\sqrt{n})$ or $O(n)$, enabling long-document processing.

Research Synthesis

The through-line from Sutskever (2014) to GPT-4 (2023) is continuous: each advance built on the context-compression-and-selective-retrieval insight.

Attention is the core operation that scales. Recurrence, position-wise FFN, and other components are architectural choices around this core.

The bottleneck is never fully solved — it merely moves: from fixed context vectors, to quadratic attention over long documents, to latency constraints at inference scale.

Further Reading

- Sutskever, Vinyals & Le (2014). Sequence to Sequence Learning with Neural Networks. NeurIPS.
- Bahdanau, Cho & Bengio (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.
- Luong, Pham & Manning (2015). Effective Approaches to Attention-based Neural Machine Translation. EMNLP.
- Vinyals, Fortunato & Jaitly (2015). Pointer Networks. NeurIPS.
- Wu et al. (2016). Google's Neural Machine Translation System. arXiv.
- Tu et al. (2016). Modeling Coverage for Neural Machine Translation. ACL.
- See, Liu & Manning (2017). Get To The Point: Summarization with Pointer-Generator Networks. ACL.
- Vaswani et al. (2017). Attention Is All You Need. NeurIPS.
- Child et al. (2019). Generating Long Sequences with Sparse Transformers. arXiv.

The future of AI is bright, and you can be part of it!